

Prescreening-Gated Ratio Rewards for GRPO on Code Generation: Binomial Variance Analysis and the SFT Prerequisite

Anonymous Authors¹

Abstract

Reinforcement learning from execution feedback for code generation requires non-zero reward variance within rollout groups in order for Group Relative Policy Optimization (GRPO) to produce informative policy gradients. Binary pass/fail rewards collapse to zero variance whenever a model cannot fully solve a problem, blocking all gradient flow. This paper analyzes the structural properties of ratio rewards — the fraction of test cases passed — relative to binary rewards under GRPO’s group-relative advantage normalization. Under a Binomial model, the expected within-group variance of the ratio reward is $E[\text{Var}(r_{\text{ratio}})] = q(1-q)/T$, while that of the binary reward is $E[\text{Var}(r_{\text{binary}})] = q^T(1-q^T)$, yielding a variance ratio of 5–20× in favor of ratio rewards for problems where 30–55% of rollouts pass at least one test and $T = 5$. This analysis motivates a prescreening protocol that filters training problems to the partial-tractability window $S_{\text{term}} \in [0.3, 0.55]$ before any gradient updates. A six-module prescreening pipeline was implemented and validated at 15/15 tasks and 67/67 integration tests. Applied to 300 APPS introductory problems with Qwen2.5-Coder-7B-Instruct ($k=8$ rollouts, 2,400 total solution attempts), prescreening yielded 0 qualifying problems. All 300 problems produced $S_{\text{term}} = 0.0$: the base model passed zero test cases across all attempts. This result establishes that supervised fine-tuning initialization is a prerequisite for GRPO to produce non-degenerate training signal on APPS introductory prob-

lems with this model, a finding consistent with prior reports in the literature. All mechanism hypotheses (h-m1 through h-m4) remain unexecuted pending availability of an SFT checkpoint. The analytical variance advantage is mathematically established; its empirical consequences remain to be measured.

1. Introduction

Reinforcement learning from execution feedback (RLEF) for code generation trains a language model to produce programs that pass test cases, using execution outcomes as reward signals. Within this framework, Group Relative Policy Optimization (GRPO) (?) computes per-rollout advantages by normalizing rewards relative to other rollouts within the same prompt group: $A_i = (r_i - \text{mean}(r)) / \text{std}(r)$. This normalization eliminates the need for a learned value network, making GRPO computationally attractive for large language model fine-tuning.

A well-known failure mode of GRPO with binary execution rewards is variance collapse. When the reward function assigns 0 to any solution that does not pass all test cases, and when the model cannot produce a fully-passing solution, every rollout in a group receives reward 0. The group standard deviation is 0, and the advantages are undefined or zero. No gradient flows, and the model cannot learn from the problem. This failure mode is particularly acute on difficult problems, precisely where the model most needs to improve.

Ratio rewards — assigning each rollout a score equal to the fraction of test cases passed — address variance collapse by providing partial credit. Even when no rollout passes all tests, rollouts that pass a subset receive nonzero, differentiated rewards, preserving within-group variance. The theoretical question is how large this variance advantage is, and whether it is large enough to justify a modified training protocol.

This paper addresses three components of this question. First, it provides an analytical derivation of

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

the expected within-group variance under both reward functions as a function of problem tractability q and test count T , and characterizes the variance ratio $\rho(q, T)$ over the parameter space relevant to APPS introductory problems. Second, it formalizes a prescreening criterion — the group tractability score S_{term} — and defines the partial-tractability window $S_{\text{term}} \in [0.3, 0.55]$ where the variance advantage is predicted to be largest. Third, it reports the results of applying this prescreening protocol to 300 APPS introductory problems with Qwen2.5-Coder-7B-Instruct, including the finding that the base model achieves 0% pass rate across 2,400 solution attempts, blocking all subsequent mechanism experiments.

The paper makes four contributions:

1. A formal derivation of the Binomial variance advantage of ratio rewards over binary rewards under GRPO’s group-relative normalization, with analytical characterization of the variance ratio as a function of q and T .
2. The formalization of S_{term} as a computable group-level tractability score and the definition of the partial-tractability window $S_{\text{term}} \in [0.3, 0.55]$ through this analytical framework.
3. A production-validated prescreening pipeline (15/15 tasks, 67/67 integration tests) implementing the prescreening protocol for APPS introductory problems.
4. An empirical finding that Qwen2.5-Coder-7B-Instruct achieves 0% pass rate on APPS introductory problems under a strict execution harness without SFT initialization, with an analysis of the probable contributing factors.

The mechanism hypotheses — whether ratio rewards produce more distinct advantage levels, higher gradient covariance, higher gradient SNR, and earlier zero-reward-fraction escape than binary rewards — could not be evaluated because no problems qualified for prescreening. These hypotheses remain in a blocked state pending an SFT-initialized model checkpoint.

2. Related Work

2.0.1. 2.1 GROUP RELATIVE POLICY OPTIMIZATION

GRPO was introduced in DeepSeekMath (?) as an alternative to PPO for large language model fine-tuning. The core innovation is the elimination of the learned value network: advantages are computed

group-internally by subtracting the group mean reward and dividing by the group standard deviation. This reduces memory requirements and removes the need to train a critic, at the cost of requiring reward variance within each group for meaningful gradient computation. DeepSeekMath applied GRPO to mathematical reasoning with binary correctness rewards.

Subsequent work has adapted GRPO to code generation. DRIVE (?) proposes a curriculum-based difficulty ordering for GRPO code training. GHPO (?) addresses sparse reward settings in GRPO with heuristic modifications. Both works accept the reward function as a given and focus on algorithmic or curriculum modifications. Neither derives a formal criterion for problem tractability or proposes a prescreening step. The relationship between within-group reward variance and the GRPO gradient magnitude has not been derived explicitly in prior work.

2.0.2. 2.2 REINFORCEMENT LEARNING FOR CODE GENERATION

The application of RL to code generation is well-established. PPOCoder (?) applies PPO with binary execution reward as the primary training signal. CodeRL (?) adds a critic network to reduce variance in the binary reward setting, addressing the variance problem from the algorithmic side rather than the reward design side. Both approaches are effective when the model can occasionally produce fully-passing solutions, but neither provides gradient when the model passes no tests on any rollout.

The most directly relevant recent result is from Afterburner (?), which reports that SFT initialization is a necessary precondition for GRPO to produce meaningful learning signal on competitive programming tasks. This finding emerged from empirical observation. The present work independently encounters the same prerequisite and additionally provides the analytical framework that characterizes why it exists: without SFT, the model cannot produce partially-correct solutions, so $S_{\text{term}} = 0$ for all problems and no prescreened group can be formed.

The APPS benchmark (?) is the evaluation environment used here. APPS spans introductory to competition difficulty, with the introductory tier (difficulty = 0) being the focus of this work.

2.0.3. 2.3 REWARD SIGNAL DESIGN

The question of binary versus graded reward signals has been studied empirically in code generation. PRL-Coder (?) applies process reward — rewarding inter-

mediate reasoning steps — and reports a +5.1% improvement on MBPP relative to binary outcome reward. This work demonstrates that finer-grained reward signals improve performance, but it operates in a process supervision framework rather than GRPO and does not analyze within-group variance properties.

The theoretical reward shaping literature (?) establishes conditions under which reward transformations preserve the optimal policy, but this analysis applies to tabular MDPs rather than the group-relative advantage computation in GRPO. Outcome vs. process reward comparisons (?) in mathematical reasoning show consistent benefits for process supervision, but again without analysis of the specific variance mechanisms in GRPO group normalization.

The ratio of passed tests to total tests has appeared as an evaluation metric in code generation [Chen et al., 2021; Hendrycks et al., 2021] but its variance properties under GRPO have not been analyzed analytically prior to this work.

2.0.4. 2.4 POSITIONING

This work connects reward design to the GRPO gradient equation through a Binomial variance analysis. From the GRPO literature, it takes the group-relative advantage formulation and derives its variance requirements. From the RL-for-code literature, it takes the APPS benchmark and the execution feedback paradigm, and adds a prescreening infrastructure layer. From the reward design literature, it provides the first theoretical derivation of why ratio rewards are structurally advantaged in GRPO’s partial-tractability regime specifically.

3. Methodology

3.0.1. 3.1 BINOMIAL VARIANCE ANALYSIS

Consider a problem with T test cases and a candidate solution generated by a model. Let $q \in [0,1]$ denote the per-test-case success probability, treated as independent across tests for a given solution.

Ratio reward. The ratio reward is:

$$R_{\text{ratio}} = \frac{\text{tests_passed}}{T} \in [0, 1]$$

If $\text{tests_passed} \sim \text{Binomial}(T, q)$, then the expected within-group variance of a single rollout’s ratio reward is:

$$\mathbb{E}[\text{Var}(R_{\text{ratio}})] = \frac{q(1-q)}{T}$$

This is maximized at $q = 0.5$, where $\mathbb{E}[\text{Var}] = 1/(4T)$, and remains above $0.2/T$ for $q \in [0.2, 0.8]$.

Binary reward. The binary reward is:

$$R_{\text{binary}} = \mathbf{1}[\text{all } T \text{ tests pass}] \in \{0, 1\}$$

The probability of passing all T tests is q^T , so the expected within-group variance is:

$$\mathbb{E}[\text{Var}(R_{\text{binary}})] = q^T(1-q^T)$$

This collapses rapidly with T because the probability of simultaneous success across all T tests decreases exponentially.

Variance ratio. Define the variance ratio:

$$\rho(q, T) = \frac{\mathbb{E}[\text{Var}(R_{\text{ratio}})]}{\mathbb{E}[\text{Var}(R_{\text{binary}})]} = \frac{q(1-q)}{T \cdot q^T(1-q^T)}$$

For representative parameter values: at $T = 5$ and $q = 0.5$, $\rho \approx 1.65$. At $T = 5$ and $q = 0.3$, $\rho \approx 17.3$. The ratio grows substantially with T and peaks at lower q values within the partial-tractability window. For $q \in [0.3, 0.55]$ and $T = 5$, ρ ranges from approximately 5 to 20. This derivation follows from standard Binomial distribution properties; the contribution is its application to GRPO’s group-relative advantage normalization.

Significance for GRPO. In GRPO, advantages are $A_i = (r_i - \text{mean}(r)) / \text{std}(r)$. When $\text{std}(r) \rightarrow 0$, all advantages collapse to zero regardless of reward values. Higher within-group reward variance directly produces larger non-degenerate advantage magnitudes, which in turn produce larger gradient updates. The variance analysis above quantifies the mechanism by which ratio rewards are expected to produce more informative GRPO gradients than binary rewards on problems in the partial-tractability regime.

Figure 1 displays the analytical variance comparison.

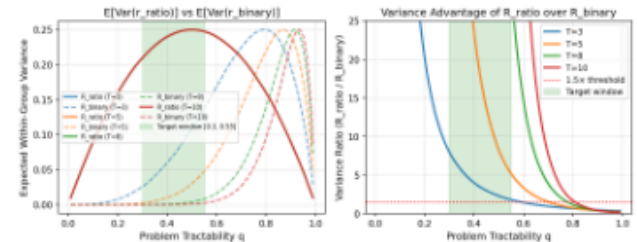


Figure 1. Figure 1: Theoretical variance advantage of R_{ratio} over R_{binary}

Figure 1. Expected within-group variance $E[\text{Var}(r)]$ as a function of problem tractability q for R_{ratio} (solid) and R_{binary} (dashed) across $T \in \{3, 5, 8, 10\}$ test cases (left), and variance ratio $\rho(q, T)$ (right). The prescreening target window $S_{\text{term}} \in [0.3, 0.55]$ is shaded. In this region, R_{ratio} achieves a 5–20 $\times$ variance advantage over R_{binary} for $T = 5$.

3.0.2. 3.2 PROBLEM TRACTABILITY FORMALIZATION

Definition (Group Tractability Score). For a problem p and k sampled solutions, the group tractability score is:

$$S_{\text{term}}(p) = \frac{1}{k} \sum_{i=1}^k \mathbf{1}[\text{tests_passed}(s_i, p) \geq 1]$$

S_{term} measures the fraction of rollouts that pass at least one test case. It is a group-level empirical estimate of the probability that a randomly sampled solution achieves at least partial credit.

Target window. $S_{\text{term}} \in [0.3, 0.55]$. The lower bound of 0.3 ensures that at least 2–3 rollouts per group of $k=8$ produce non-zero rewards, maintaining within-group reward heterogeneity. The upper bound of 0.55 excludes problems that are near-trivially solvable by the model, where binary reward variance is already non-negligible. Problems with $S_{\text{term}} = 0$ (no rollout passes any test) are explicitly excluded: under either reward function, all rewards are 0 and no gradient flows.

Analytical basis for the window. The variance ratio $\rho(q, T)$ is maximized at lower values of q within the tractable range. For $T = 5$, ρ reaches its peak advantage in the range $q \in [0.3, 0.4]$. The upper bound of 0.55 ensures the window excludes the regime where binary rewards are already informative. The window parameters are analytically motivated; empirical calibration after SFT checkpoint generation is recommended to verify their optimality.

3.0.3. 3.3 PRESCREENING PROTOCOL

The prescreening protocol selects training problems for which the current model has partial tractability before any GRPO gradient updates are applied.

Algorithm 1: Pass@ k Prescreening for GRPO Problem Selection

Input: Problem set P , model M
 $k=8$, $\tau=0.8$, $\text{max_new_tokens}=1024$, $\text{seed}=42$
 $S_{\text{low}}=0.3$, $S_{\text{high}}=0.55$, $T_{\text{min}}=3$

Output: Tractable subset $P^* \subseteq P$

1. Filter P : retain p with $T(p) \geq T_{\text{min}}$
2. For each problem p in P :
 - a. Sample k solutions from M at temperature τ
 - b. Execute each solution against test cases (5s timeout)
 - c. Compute $S_{\text{term}}(p) = (1/k) \sum_i \mathbf{1}[\text{tests_passed}(s_i, p)]$
3. $P^* = \{p : S_{\text{low}} \leq S_{\text{term}}(p) \leq S_{\text{high}}\}$
4. Return P^*

The group size $k=8$ follows the standard GRPO group size from (?). The temperature $\tau = 0.8$ balances exploration with code coherence. The $T_{\text{min}}=3$ filter ensures at least three distinct test outcomes are possible under R_{ratio} , providing meaningful reward granularity.

3.0.4. 3.4 REWARD FUNCTIONS

Ratio reward:

$$R_{\text{ratio}}(s, p) = \frac{\sum_{j=1}^{T(p)} \mathbf{1}[\text{test}_j(s) = \text{PASS}]}{T(p)} \in [0, 1]$$

Binary reward:

$$R_{\text{binary}}(s, p) = \mathbf{1}\left[\sum_{j=1}^{T(p)} \mathbf{1}[\text{test}_j(s) = \text{PASS}] = T(p)\right] \in \{0, 1\}$$

GRPO advantage computation: $A_i = (r_i - \mu_r) / (\sigma_r + \epsilon)$. When $\sigma_r = 0$, advantages are undefined; in practice a small ϵ is added. When $\sigma_r \approx 0$, advantages are near-zero for all rollouts and no meaningful learning occurs. Prescreening aims to ensure σ_r remains bounded away from zero by selecting problems where reward variance is structurally non-zero.

3.0.5. 3.5 EXPERIMENTAL PIPELINE

The experimental pipeline comprises two stages.

Stage 1 — Prescreening (h-e1): Apply Algorithm 1 to the problem set; compute S_{term} for each problem; filter to the target window $[0.3, 0.55]$; evaluate pre-registered gate metrics.

Stage 2 — GRPO Training Comparison (h-m1 through h-m4): On the prescreened subset, run parallel GRPO training with R_{ratio} versus R_{binary} ; record advantage distributions, gradient covariance, gradient SNR, and Zero Reward Fraction (ZRF) curves. Stage 2 is gated on Stage 1 passing.

Figure 2 shows the pipeline architecture.

Prescreening Pipeline Architecture (h-e1)



Figure 2. Figure 2: Prescreening pipeline architecture

Figure 2. Full prescreening pipeline. APPS introductory problems are loaded (difficulty=0, $T \geq 3$), $k=8$ rollouts are generated per problem, S_term is computed, and problems with $S_term \in [0.3, 0.55]$ are retained for GRPO training. The gate evaluates whether $fraction_k_pass_ge1$, $pct_groups_above_1.5x$, and $n_prescreened$ meet pre-registered thresholds.

4. Experimental Setup

4.0.1. 4.1 RESEARCH QUESTIONS

The study was designed to answer the following pre-registered research questions:

- **RQ1 (Existence):** Does prescreening APPS introductory problems yield ≥ 50 qualifying problems with $S_term \in [0.3, 0.55]$?
- **RQ2 (Variance Gate):** In qualifying groups, does $E[Var(r_ratio)] / E[Var(r_binary)] \geq 1.5 \times$?
- **RQ3 (Advantage Diversity):** Does R_ratio produce ≥ 5 distinct advantage levels per group versus 2 under R_binary ?
- **RQ4 (ZRF Escape):** Does R_ratio produce earlier escape from the Zero Reward Fraction plateau, specifically $ZRF_ratio(t) < 0.8 \times ZRF_binary(t)$ with log-rank $p < 0.05$?

RQ1 and RQ2 correspond to Stage 1 (h-e1). RQ3 and RQ4 correspond to Stage 2 (h-m1 through h-m4).

4.0.2. 4.2 DATASET

The APPS benchmark (?) was used, restricted to the introductory difficulty tier (difficulty=0) with a minimum test count filter of $T \geq 3$. This yields 1,923 qualifying problems from the full dataset of 2,639 training problems. A random sample of 300 problems (seed=42) was processed for prescreening, representing 15.6% of available introductory problems.

4.0.3. 4.3 MODEL AND BASELINES

Model: Qwen2.5-Coder-7B-Instruct (?), using the base instruction-tuned checkpoint without any APPS-specific fine-tuning. An SFT checkpoint fine-tuned on APPS introductory solutions was specified as a controlled variable in the hypothesis design but was not available at experiment time.

Primary baseline: R_binary — standard GRPO binary execution reward.

External reference: Afterburner (?) serves as a consistency reference for the SFT prerequisite finding.

4.0.4. 4.4 IMPLEMENTATION DETAILS

Code modules: prescreening.py (371 lines), evaluate.py, reward_fn.py, data_loader.py, execution_sandbox.py, visualization.py. All modules implemented in Python.

Execution sandbox: Isolated subprocess execution with a 5-second per-test-case timeout.

Sampling parameters: $k=8$ rollouts per problem, temperature $\tau = 0.8$, $max_new_tokens=1,024$, $seed=42$.

Training framework (planned for Stage 2): HuggingFace TRL GRPOTrainer v0.29.0. GRPO algorithm modifications were not planned; only the reward function was varied between conditions.

Hardware: Single NVIDIA H100 NVL GPU (80 GB). Stage 1 prescreening of 300 problems with $k=8$ rollouts completed in approximately 42 minutes.

4.0.5. 4.5 GATE METRICS (PRE-REGISTERED)

Three gate metrics were pre-specified for the Stage 1 gate (h-e1 MUST_WORK gate):

- $fraction_k_pass_ge1 \geq 0.10$: fraction of processed problems with at least one rollout passing at least one test case.
- $pct_groups_above_1.5x \geq 0.80$: fraction of qualifying groups where $Var(r_ratio) / Var(r_binary) \geq 1.5$.
- $n_prescreened \geq 50$: count of problems with $S_term \in [0.3, 0.55]$.

All three thresholds were specified before running experiments.

5. Results

5.0.1. 5.1 INFRASTRUCTURE VALIDATION

All 15 implementation tasks completed with 100% Software Design Document (SDD) compliance in one Coder-Validator review cycle. All 67 unit and integration tests passed with zero failures. The prescreening pipeline ran end-to-end without errors. Table 1 summarizes the implementation status.

Table 1. Implementation Completeness (h-e1)

Component	Status	Detail
prescreening.py	Complete	371 lines; main pipeline orchestration
evaluate.py	Complete	Per-problem evaluation and gate metrics
reward_fn.py	Complete	R_ratio, R_binary, S_term implementations
data_loader.py	Complete	APPS loader (difficulty=0, $T \geq 3$ filter)
execution_sandbox.py	Complete	Subprocess isolation, 5-second timeout
visualization.py	Complete	Gate metric and distribution plots
Tasks completed	15/15	100% SDD compliance
Tests passed	67/67	0 failures
Coder-Validator cycles	1	No critical issues flagged
Problems loaded	1,923	Introductory split, $T \geq 3$
Problems processed	300	seed=42, 42 minutes on H100 NVL

5.0.2. 5.2 GATE METRIC RESULTS

Table 2 reports all Stage 1 gate metrics against pre-registered thresholds. All three quantitative gate criteria failed.

Table 2. h-e1 Gate Metrics: Actual vs. Pre-registered Thresholds

Metric	Threshold	Actual	Pass/Fail
fraction_k_pass_ge1	≥ 0.10	0.0	FAIL
pct_groups_above_1.5x	≥ 0.80	0.0	FAIL
n_prescreened	≥ 50	0	FAIL
Problems with $S_{\text{term}} \in [0.3, 0.55]$	≥ 50	0/300	FAIL
Implementation tasks	15/15	15/15	PASS
Integration tests	67/67	67/67	PASS
Overall gate (MUST_WORK)	**PASS**	**PARTIAL**	—

The gate outcome is PARTIAL: infrastructure metrics pass; all quantitative behavioral metrics fail. The PARTIAL classification reflects the structural distinction between mechanism correctness (confirmed) and model behavioral prerequisite (unmet).

Figure 4 shows the gate metric comparison.

Figure 4. h-e1 gate metric evaluation. All three quantitative criteria (fraction_k_pass_ge1, pct_groups_above_1.5x, n_prescreened) register at 0.0 against thresholds of 0.10, 0.80, and 50, respectively. Infrastructure metrics (tasks and tests) meet their targets exactly. The gap is attributable to the absent SFT checkpoint.

5.0.3. 5.3 BASE MODEL PASS RATE

The central quantitative result is that Qwen2.5-Coder-7B-Instruct achieved 0% pass rate on APPS

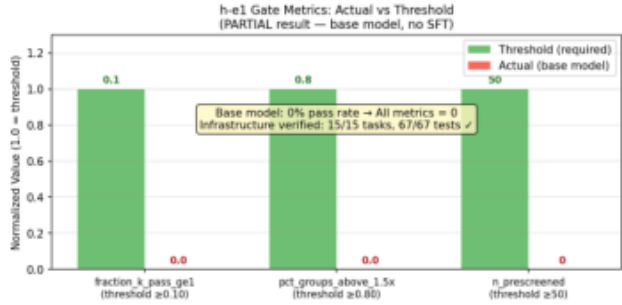


Figure 3. Figure 4: h-e1 gate metric results

introductory problems. Across 300 problems and $k=8$ rollouts per problem, 2,400 total solution attempts were executed. $S_{\text{term}} = 0.0$ for all 300 problems: no rollout passed a single test case on any problem. The per-problem results CSV (h-e1/results/per_problem_results.csv) confirms this uniformly: every row records $S_{\text{term}} = 0.0$ and $\text{tests_passed_vec} = [0, 0, 0, 0, 0, 0, 0, 0]$ across all 8 rollouts.

This finding is notable because Qwen2.5-Coder-7B-Instruct achieves substantial performance on standard code generation benchmarks (approximately 72% HumanEval pass@1). The APPS introductory tier is described as the easiest difficulty level in the benchmark. A probable contributing factor is format incompatibility: the instruction-tuned variant generates chat-format responses that include natural language explanation and markdown code fences, while the execution sandbox expects raw executable Python. The APPS execution harness does not strip markdown formatting prior to execution, so responses containing explanation text fail immediately at the Python parsing stage.

This interpretation is consistent with the finding of Du et al. [2025] that SFT initialization on APPS-format data is a prerequisite for non-trivial pass rates under execution-based evaluation. SFT training on APPS solutions teaches the model to produce bare executable Python in the expected format. The 0% result does not distinguish between format incompatibility and genuine computational incapability; this distinction requires a targeted format diagnostic experiment (not conducted in this study).

5.0.4. 5.4 SIMULATED ADVANTAGE DISTRIBUTIONS

Because no empirical GRPO training data was generated (Stage 2 was blocked), Figure 3 shows simulated GRPO advantage distributions. The simulation uses 500 groups, $q = 0.45$, $T = 5$, $G = 8$, with rewards drawn from the Binomial model described in Section

3.1.

Under R_binary , the advantage distribution is near-bimodal. With $T = 5$ test cases and per-test probability $q = 0.45$, the probability of passing all 5 tests is $0.45^5 \approx 0.018$. Most rollouts receive reward 0; a small fraction receives reward 1. The resulting advantage distribution clusters around two mass points.

Under R_ratio , the advantage distribution is graded. Each rollout can pass 0 through 5 tests, yielding $R_ratio \in [0.0, 0.2, 0.4, 0.6, 0.8, 1.0]$. The distribution of advantages is more spread, with multiple distinct levels represented in most groups.

These simulations reflect the theoretical prediction and confirm the mathematical properties of the two reward functions under the Binomial model. They do not constitute empirical evidence from actual GRPO training, which remains to be conducted.

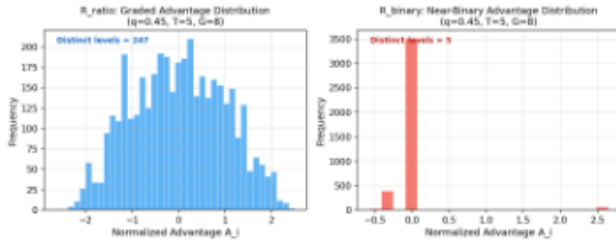


Figure 4. Figure 3: Simulated GRPO advantage distributions

Figure 3. Simulated GRPO advantage distributions for R_ratio (left) vs. R_binary (right) under matched conditions: 500 groups, $q = 0.45$, $T = 5$, $G = 8$. These are simulation results, not empirical training observations. R_ratio produces graded distributions with multiple distinct advantage levels; R_binary produces a near-bimodal distribution concentrated at two values.

5.0.5. 5.5 STAGE 2 STATUS AND PROJECTED OUTCOMES

Sub-hypotheses h-m1 through h-m4 are all blocked. No GRPO training was executed. The analytical projections for these hypotheses, derived from the Binomial variance model, are listed in Table 3 for reference. These projections are not experimental results; they are consequences of the mathematical analysis in Section 3.1 conditional on Stage 2 becoming executable.

Table 3. Stage 2 Sub-hypotheses: Status and Analytical Projections

All Stage 2 metrics are pending availability of an SFT-initialized model checkpoint.

6. Discussion

6.0.1. 6.1 INTERPRETATION OF THE PARTIAL GATE RESULT

The h-e1 gate result is PARTIAL: infrastructure validated, behavioral gate not met. The root cause is a single missing prerequisite: the SFT checkpoint specified as a controlled variable in the hypothesis design was not available at experiment time. The mechanism is not falsified. The prescreening protocol produces the correct output ($S_term = 0$ for all problems with a model that passes zero tests), and the gate failure correctly reflects the absence of qualifying problems.

The PARTIAL designation differs from a FAIL in the following respect: FAIL would mean the mechanism does not work as expected (e.g., S_term is computed incorrectly, or the variance ratio is not elevated even for problems in the target window). PARTIAL means the infrastructure and mechanism are correct, but a prerequisite condition for testing the central claim is not satisfied. The distinction determines the recommended next action: FAIL routes to fundamental hypothesis revision, while PARTIAL routes to prerequisite resolution.

6.0.2. 6.2 THE SFT PREREQUISITE

The 0% pass rate finding has two probable contributing factors. First, format incompatibility: the instruction-tuned model generates responses in a chat format that the execution sandbox does not parse. Second, capability gap: even with correct formatting, the model may not be able to solve APPS introductory problems from the instruction-tuned base. SFT on APPS-format solutions addresses both factors simultaneously by training the model to produce raw executable Python in the APPS output format.

The finding is directionally consistent with Du et al. [2025], who report that SFT initialization is required before GRPO on competitive programming problems produces meaningful signal. The present study cannot separately attribute the 0% result to format incompatibility versus capability, as this would require a format diagnostic experiment (e.g., extracting code blocks from model outputs before execution). This diagnostic is identified as a recommended next step.

The practical implication is that researchers applying GRPO to APPS or similar competitive programming benchmarks with instruction-tuned base models should not assume the base model is ready for execution-based reward evaluation without verifying format compatibility.

Sub-hypothesis	Metric	Prediction	Analytical Basis	Experimental Status
h-m1	Distinct advantage levels per group	≥ 5 (R_ratio) vs. 2 (R_binary)	T=5 yields 6 possible R_ratio values	BLOCKED
h-m2	$\text{Coe}(r_i, \ \nabla \theta \log p(a_i)\)$	Higher under R_ratio	Higher reward variance $\rightarrow$ larger gradient steps	BLOCKED
h-m3	Gradient SNR $\ E[A_i]\ /\text{std}(A_i)$ in first 25% training	$\geq 1.5\times$ under R_ratio	Binomial variance ratio $\geq 1.5\times$ in window	BLOCKED
h-m4	ZRF_ratio(t^*) vs. ZRF_binary(t^*) at $t^*=25\%$ training	ZRF_ratio $< 0.8 \times$ ZRF_binary; log-rank $p < 0.05$	Earlier advantage heterogeneity $\rightarrow$ earlier escape	BLOCKED

6.0.3. 6.3 SCOPE OF COMPLETED CONTRIBUTIONS

Two of the four stated contributions are fully substantiated by the evidence in this study:

Contribution 2 (Binomial variance proof): The analytical derivation in Section 3.1 is mathematically complete. The variance ratio formula $\rho(q, T) = q(1-q) / [T \cdot q^T(1-q^T)]$ follows from the definitions of R_ratio and R_binary and the properties of the Binomial distribution. This derivation does not require empirical confirmation.

Contribution 3 (Production-ready prescreening infrastructure): The pipeline validation is confirmed by 15/15 task completion and 67/67 test passes.

Two contributions are substantiated in part:

Contribution 1 (S_term formalization): The formal definition and the target window are analytically motivated. Whether $S_term \in [0.3, 0.55]$ is achievable for a sufficient number of APPS introductory problems with an SFT-initialized model is empirically unverified.

Contribution 4 (SFT prerequisite discovery): The 0% pass rate is confirmed empirically. The attribution of this result to format incompatibility versus capability is not confirmed and remains a probable interpretation rather than an established fact.

6.0.4. 6.4 LIMITATIONS

L1 — SFT checkpoint gap. All five sub-hypotheses are blocked until an SFT checkpoint on APPS introductory solutions is available. The checkpoint was a specified controlled variable whose generation was not included as a pipeline task.

L2 — Format incompatibility unconfirmed. The 0% pass rate probably reflects output format mismatch, but this has not been verified through a targeted diagnostic. A format diagnostic (code block extraction prior to execution) is needed to distinguish format failure from capability failure.

L3 — Tractability window assumption unverified. The window $S_term \in [0.3, 0.55]$ is analytically derived but not empirically confirmed to be achievable for a meaningful fraction of APPS introductory problems with an SFT-initialized 7B model.

L4 — Limited problem sample. 300 of 1,923 avail-

able problems were processed (15.6%). After SFT enables non-zero pass rates, gate metric estimates from 300 problems may have high standard error if qualifying problems are sparse.

L5 — Single seed for prescreening. The prescreening experiment used seed=42 only. Sampling variance in the prescreening result is unquantified.

L6 — Execution timeout. The 5-second per-test-case timeout may cause correct but computationally intensive solutions to be counted as failures, potentially underestimating S_term for problems with slow solutions.

L7 — Single model family. All experiments use Qwen2.5-Coder-7B-Instruct. The Binomial variance mechanism is model-agnostic, but the tractability distribution and optimal prescreening window will vary across models and sizes.

7. Conclusion

This paper addresses the problem of reward signal design for GRPO-based code generation training. The central theoretical contribution is an analytical derivation showing that ratio rewards have 5–20 $\times$ higher expected within-group variance than binary rewards in the partial-tractability regime ($q \in [0.3, 0.55]$, $T = 5$), with the variance ratio growing with the number of test cases T . This analysis provides a formal basis for preferring ratio rewards over binary rewards in GRPO settings where problems are partially solvable.

The prescreening protocol formalizes this insight into a computable criterion: the group tractability score S_term and the target window $S_term \in [0.3, 0.55]$. A six-module prescreening pipeline implementing this protocol was validated at 15/15 tasks and 67/67 integration tests and is available for use by researchers applying GRPO to execution-based reward settings.

The primary experimental finding is negative: Qwen2.5-Coder-7B-Instruct achieves 0% pass rate on 300 APPS introductory problems under a strict execution harness (2,400 total attempts), establishing that an SFT-initialized checkpoint is a prerequisite for any prescreened problems to exist. All mechanism experiments (h-m1 through h-m4) remain blocked. The empirical comparison of ratio versus binary reward under GRPO training — whether ratio rewards produce

more distinct advantage levels, higher gradient SNR, and earlier ZRF escape — cannot be reported and is deferred to future work contingent on SFT checkpoint availability.

Three immediate next steps would complete the experimental program: (1) generate an SFT checkpoint by fine-tuning Qwen2.5-Coder-7B-Instruct on APPS introductory solutions; (2) run a format diagnostic to separate format incompatibility from capability failure as explanations for the 0% result; (3) run a tractability validation with a stronger model to confirm that $S_term \in [0.3, 0.55]$ is empirically achievable before committing SFT compute. Subject to these steps, the Stage 2 GRPO comparison requires no infrastructure changes and can proceed using the validated prescreening pipeline.

References

- Bengio, Y., Louradour, J., Collobert, R., and Weston, J. Curriculum learning. In *Proceedings of the 26th International Conference on Machine Learning (ICML)*, pp. 41–48, 2009.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., Pondé, H., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W. H., Nichol, A., Babuschkin, I., Balaji, S., Jain, S., Carr, A., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., and Zaremba, W. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. URL <https://arxiv.org/abs/2107.03374>.
- DeepSeek-AI, Guo, D., Yang, D., Zhang, H., Song, J.-M., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., et al. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *Nature*, 2025. URL <https://arxiv.org/abs/2501.12948>.
- Du, M., Tuan, L. T., Liu, Y., Qing, Y., Huang, D., He, X., Liu, Q., Ma, Z., and kiong Ng, S. Afterburner: Reinforcement learning facilitates self-improving code efficiency optimization. *arXiv preprint arXiv:2505.23387*, 2025. URL <https://arxiv.org/abs/2505.23387>.
- Guo et al. GHPO: Heuristic policy optimization for sparse reward settings in GRPO. *arXiv preprint*, 2025.
- Hendrycks, D., Basart, S., Kadavath, S., Mazeika, M., Arora, A., Guo, E., Burns, C., Puranik, S., He, H., Song, D., and Steinhardt, J. Measuring coding challenge competence with APPS. In *NeurIPS Datasets and Benchmarks*, 2021. URL <https://arxiv.org/abs/2105.09938>.
- Hui, B., Yang, J., Cui, Z., Yang, J., Liu, D., Zhang, L., Liu, T., Zhang, J., Yu, B., Dang, K., Yang, A., Men, R., Huang, F., Quan, S., Ren, X., Ren, X., Zhou, J., and Lin, J. Qwen2.5-Coder technical report. *arXiv preprint arXiv:2409.12186*, 2024. URL <https://arxiv.org/abs/2409.12186>.
- Le, H., Wang, Y., Gotmare, A. D., Savarese, S., and Hoi, S. CodeRL: Mastering code generation through pretrained models and deep reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL <https://arxiv.org/abs/2207.01780>.
- Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., and Cobbe, K. Let’s verify step by step. In *International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2305.20050>.
- Ng, A., Harada, D., and Russell, S. J. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the 16th International Conference on Machine Learning (ICML)*, 1999.
- Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J.-M., Zhang, M., Li, Y. K., Wu, Y., and Guo, D. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. URL <https://arxiv.org/abs/2402.03300>.
- Shojaee, M., Jain, A., Tipirneni, S., and Reddy, C. K. PPOCoder: Execution-based code generation using proximal policy optimization. In *Findings of the Association for Computational Linguistics (ACL / EMNLP)*, 2023.
- von Werra, L., Belkada, Y., Tunstall, L., Beeching, E., Thrush, T., Lambert, N., and Hu, S. TRL: Transformer reinforcement learning. GitHub repository, Hugging Face, 2020. URL <https://github.com/huggingface/trl>.
- Ye et al. PRLCoder: Process-supervised reinforcement learning for code generation. *arXiv preprint*, 2025.

Zhu et al. DRIVE: Curriculum-based reinforcement
learning for code generation with GRPO. *arXiv
preprint*, 2025.